

节点压缩包格式规范模板（NEM）

统一节点压缩包的目录结构、文件内容、阅读规则、AI 接手规则与版本管理规则

版本：v1.0

适用对象：所有 NEM 节点压缩包

用途：统一节点压缩包的目录结构、文件内容、阅读规则、AI 接手规则与版本管理规则

原则：PDF 给人看，Markdown / JSON / YAML / TXT 给 AI 和工程系统看

一、规范目的

NEM 节点压缩包不是普通资料包，而是一个可以被人理解、被 AI 接手、被工程系统执行、被未来版本继续升级的节点容器。

它的目标是让任何人或 AI 拿到压缩包后，都可以快速判断：

- 这个节点是什么；
- 为什么这个节点重要；
- 当前状态在哪里；
- 下一步应该做什么；
- 哪些文件给人看；
- 哪些文件给 AI 看；
- 哪些文件用于工程运行；
- 如何验收节点是否完成；
- 如何继续升级这个节点。

该规范适用于所有 NEM 节点，不绑定任何具体项目、具体编号或具体任务。

二、核心设计原则

人类入口与 AI 入口分离

人类需要快速理解意义、结构、优先级和路线。

AI 需要准确读取状态、规则、文件路径、执行边界和验收条件。

因此，压缩包必须同时包含：

- 给人看的 PDF 说明书；

- 给人快速浏览的 Markdown 入口；
- 给 AI 接手的 Markdown 指令；
- 给机器读取的 Manifest 文件；
- 给工程系统使用的模板、命令、日志和报告目录。

节点不是一次性任务

NEM 节点是可进化模块。

它可以被新增、升级、重构、分叉、合并和采纳。

压缩包的设计不能只服务于当前一次执行，而要服务于长期演化。

文件结构必须稳定

目录结构稳定，AI 才能长期接手。

文件命名稳定，工程系统才容易读取。

规则稳定，团队协作才不会混乱。

PDF 是认知入口，不是执行入口

PDF 的作用是让人快速理解节点。

AI 和工程执行不应依赖 PDF，而应依赖 Markdown、JSON、YAML、TXT、日志和模板文件。

三、压缩包总目录模板

下面是标准 NEM 节点压缩包的推荐目录结构。

```
NEM_Node_Package/  
|  
|-- 00_README_HUMAN.md  
|-- 00_README_AI.md  
|-- 00_PACKAGE_MANIFEST.json  
|-- 00_NODE_STATUS.md  
|  
|-- pdf/  
|   |-- readable_overview.pdf  
|  
|-- nem/  
|   |-- node_definition.md  
|  
|-- aep/  
|   |-- aep_template.md  
|   |-- active_aep.md  
|   |-- completed/  
|  
|-- gate/  
|   |-- gate_definition.md  
|   |-- current_gate.md  
|   |-- gate_review_template.md  
|  
|-- rules/
```

```

|-- package_rules.md
|-- ai_execution_rules.md
|-- naming_convention.md
|-- versioning_rules.md
|-- adoption_rules.md

|-- templates/
|-- config.template.json
|-- registry.template.jsonl
|-- queue.template.jsonl
|-- report.template.md
|-- failure_report.template.md
|-- decision_log.template.md

|-- commands/
|-- run_commands.md
|-- check_progress_commands.md
|-- stop_resume_commands.md
|-- git_commands.md

|-- reports/
|-- README_reports.md

|-- logs/
|-- README_logs.md

|-- decisions/
|-- decision_log.md

|-- backlog/
|-- backlog.md

-- archive/
|-- README_archive.md

```

说明：该目录只提供结构模板，不包含任何具体任务编号。实际项目可以替换文件名中的通用描述，但不应破坏目录层级和阅读规则。

四、三层结构

标准 NEM 压缩包由三层组成。

层级	面向对象	核心作用	主要文件
Human Layer	人类	理解节点意义、背景、路线和验收标准	PDF、README_HUMAN、节点总说明
AI Layer	AI Agent	接手节点、读取状态、执行 AEP、更新记录	README_AI、Manifest、Status、NEM、AEP、Gate、Rules
Runtime Layer	工程系统	提供运行配置、命令、日志、报告和模板	Templates、Commands、Logs、Reports

这三层缺一不可。

只有 Human Layer，节点会变成宣传资料。

只有 AI Layer，节点缺少人类理解入口。

只有 Runtime Layer，节点缺少战略定位和进化结构。

五、根目录文件规范

00_README_HUMAN.md

用途：给人类快速理解压缩包。

建议包含：

- 节点名称；
- 一句话目标；
- 当前状态；
- 人类阅读顺序；
- AI 阅读顺序；
- 当前最重要动作；
- 需要注意的风险。

推荐结构：

```
# 节点压缩包说明
## 节点一句话目标
## 当前状态
## 人类阅读顺序
## AI 阅读顺序
## 当前最重要动作
## 注意事项
```

00_README_AI.md

用途：给 AI Agent 的接手说明。

建议包含：

- AI 的角色定位；
- 必须先读取的文件；
- 禁止事项；
- 执行顺序；
- 更新规则；
- 交付规则；
- 失败处理规则。

关键原则：AI 不应拿到压缩包后重新设计整个节点，而应先读取状态、规则、当前 Gate 和当前 AEP，再按既有结构推进。

推荐结构：

```
# AI 接手说明

## 你的角色

## 首先读取

## 执行边界

## 禁止事项

## 每次执行后必须更新

## 失败时如何处理
```

00_PACKAGE_MANIFEST.json

用途：机器可读的压缩包索引。

推荐字段：

```
{
  "package_type": "NEM Node Package",
  "package_name": "<node package name>",
  "project": "<project name>",
  "version": "<version>",
  "status": "<draft | active | hold | adopt | archived>",
  "human_entry": "pdf/readable_overview.pdf",
  "ai_entry": "00_README_AI.md",
  "main_node_file": "nem/node_definition.md",
  "status_file": "00_NODE_STATUS.md",
  "current_gate": "gate/current_gate.md",
  "active_aep": "aep/active_aep.md",
  "rules_dir": "rules/",
  "templates_dir": "templates/",
  "commands_dir": "commands/",
  "reports_dir": "reports/",
  "logs_dir": "logs/"
}
```

Manifest 的作用是让 AI、脚本或未来工具不需要猜目录结构。

00_NODE_STATUS.md

用途：节点当前状态文件，也可以理解为节点心跳。

建议包含：

- 当前状态；
- 当前 Gate；
- 当前 AEP；
- 当前完成度；

- 当前阻塞；
- 下一步动作；
- 最近更新时间；
- 最近一次决策。

推荐结构：

```
# 节点状态
## 当前状态
## 当前 Gate
## 当前 AEP
## 当前完成度
## 当前阻塞
## 下一步动作
## 最近更新时间
## 最近一次决策
```

六、PDF 文件规范

目录

pdf/

文件

readable_overview.pdf

定位

PDF 是给人看的说明版文件，不是 AI 执行入口。

PDF 应回答的问题

- 这个节点是什么；
- 为什么要做这个节点；
- 它在整个工程链中的位置；
- 它包含哪些模块；
- 当前状态是什么；
- 未来如何验收；
- 完成后会带来什么能力。

PDF 不应承担的内容

- 不应作为唯一执行依据；
- 不应塞入过多机器配置；
- 不应替代 Markdown 文件；
- 不应成为日志文件；
- 不应承担状态更新职责。

PDF 是给人看的地图，Markdown 和 JSON 才是给 AI 与系统使用的路线和接口。

七、nem/ 目录规范

目录

nem/

核心文件

node_definition.md

用途

该文件是节点总定义，负责描述：

- 节点是什么；
- 节点为什么存在；
- 节点边界是什么；
- 节点不负责什么；
- 节点由哪些 AEP 构成；
- 节点通过哪些 Gate 验收；
- 节点最终完成定义是什么。

推荐结构

```
# 节点总定义
## 节点一句话定义
## 节点定位
## 节点目标
## 节点边界
## 不负责范围
```

```
## 组成模块

## Gate 验收路径

## PoEW 工程工作量证明

## 完成定义
```

规则

- NEM 文件不应频繁重写；
- NEM 文件用于稳定定义节点边界；
- 新增功能应优先新增 AEP，而不是随意改写节点总定义；
- 节点重构需要写入 Decision Log。

八、aep/ 目录规范

目录

```
aep/
```

用途

AEP 是节点内部的工程原子包。

每个 AEP 应是可以单独执行、单独验收、单独复盘的最小工程做功单元。

推荐文件

```
aep_template.md
active_aep.md
completed/
```

AEP 文件推荐结构

```
# AEP 工程原子包

## 目标

## 输入

## 输出

## 执行范围

## 不做什么

## 操作步骤

## 验收标准

## 失败处理

## 需要更新的文件

## 完成记录
```


规则

- 每次只应有一个主要 active AEP；
- 完成后的 AEP 可以移入 completed；
- 新增 AEP 必须说明输入、输出和验收标准；
- AEP 不能无限扩张，过大的 AEP 应拆分；
- AEP 不应重新定义整个 NEM。

九、gate/ 目录规范

目录

gate/

用途

Gate 是进化闸门，用于判断节点是否可以进入下一阶段。

Gate 不是任务，而是验收标准。

任务完成不等于 Gate 通过，Gate 通过必须有证据。

推荐文件

gate_definition.md
current_gate.md
gate_review_template.md

Gate 文件推荐结构

```
# Gate 验收文件  
  
## Gate 名称  
  
## 进入条件  
  
## 验收标准  
  
## 必须提供的证据  
  
## 失败条件  
  
## 通过后的状态变化  
  
## 评审记录
```

规则

- Gate 必须基于结果，而不是感觉；

- Gate 通过必须写入 Decision Log ；
- Gate 未通过时， 应进入修复或补充 AEP ；
- Gate 不能被 AI 擅自跳过 ；
- 进入 ADOPT 状态必须经过明确 Gate 记录。

十、rules/ 目录规范

目录

rules/

用途

Rules 是压缩包的内部宪法。

它规定 AI、团队成员和工程系统如何使用这个包。

推荐文件

package_rules.md
ai_execution_rules.md
naming_convention.md
versioning_rules.md
adoption_rules.md

应包含的规则

- 目录不能随意更改 ；
- 文件命名必须稳定 ；
- AI 必须先读 Manifest 和 Status ；
- AI 每次执行后必须更新状态文件 ；
- 重大修改必须写入 Decision Log ；
- Gate 通过必须有证据 ；
- PDF 只作为人类说明， 不作为执行依据 ；
- 模板文件不能被误当成真实运行文件 ；
- 日志和报告不能覆盖历史版本。

十一、templates/ 目录规范

目录

templates/

用途

Templates 是复用资产。

它们用于生成配置、注册表、任务队列、报告、失败记录和决策日志。

推荐文件

```
config.template.json
registry.template.jsonl
queue.template.jsonl
report.template.md
failure_report.template.md
decision_log.template.md
```

规则

- 模板文件必须使用 .template 标记；
- 模板不应直接作为生产文件运行；
- 模板字段应包含占位符；
- 模板修改必须保持向后兼容；
- 模板应尽量通用，便于其他 NEM 复用。

十二、commands/ 目录规范

目录

commands/

用途

Commands 存放可复制执行的命令说明。

推荐文件

```
run_commands.md
check_progress_commands.md
stop_resume_commands.md
git_commands.md
```

规则

- 命令必须带说明；
- 危险命令必须标注风险；

- 删除、覆盖、重置类命令必须谨慎；
- 命令应优先使用相对路径；
- 不应包含私密密钥、密码或服务器敏感信息。

十三、reports/ 与 logs/ 目录规范

reports/

用于存放人类可读的运行总结、实验总结、节点评审和阶段报告。

推荐保留：

reports/README_reports.md

logs/

用于存放运行过程日志、错误日志、长时运行记录和调试输出。

推荐保留：

logs/README_logs.md

规则

- 报告用于复盘，日志用于追踪；
- 报告可以整理，日志应尽量保留原始性；
- 重要运行必须生成报告；
- 失败运行必须保留日志；
- 历史报告和日志不应被覆盖。

十四、decisions/ 目录规范

目录

decisions/

核心文件

decision_log.md

用途

Decision Log 是节点的决策记忆。

应记录：

- 为什么改变方向；
- 为什么新增 AEP；
- 为什么通过或拒绝 Gate；
- 为什么修改目录或模板；
- 为什么从 ACTIVE 进入 ADOPT；
- 为什么归档某个文件或方案。

推荐结构：

```
# Decision Log
## 决策时间
## 决策内容
## 决策原因
## 影响范围
## 替代方案
## 后续动作
```

十五、backlog/ 目录规范

目录

```
backlog/
```

核心文件

```
backlog.md
```

用途

Backlog 用于存放暂时不执行、但未来可能进入 AEP 的想法。

规则：

- Backlog 不是当前任务；
- Backlog 不应干扰 active AEP；
- Backlog 中的事项进入执行前，必须转化为 AEP；
- Backlog 应定期清理。

十六、archive/ 目录规范

目录

archive/

用途

Archive 用于保存过时版本、旧方案、废弃模板、已替换报告等。

规则：

- 不要直接删除重要历史文件；
- 归档文件应说明归档原因；
- 已归档内容不再作为当前执行依据；
- AI 不应优先读取 archive，除非被明确要求复盘历史。

十七、统一读取规则

人类读取顺序

pdf/readable_overview.pdf
00_README_HUMAN.md
nem/node_definition.md
gate/current_gate.md
commands/run_commands.md

AI 读取顺序

00_README_AI.md
00_PACKAGE_MANIFEST.json
00_NODE_STATUS.md
rules/ai_execution_rules.md
nem/node_definition.md
gate/current_gate.md
aep/active_aep.md
templates/

工程执行读取顺序

commands/run_commands.md
templates/config.template.json
templates/registry.template.jsonl
templates/queue.template.jsonl
reports/report.template.md
logs/README_logs.md

十八、状态规范

推荐使用以下节点状态：

状态	含义
DRAFT	节点草稿阶段，结构尚未确定
ACTIVE	节点正在执行
HOLD	节点暂停，等待条件或数据
REVIEW	节点等待验收
ADOPT	节点通过验收，成为正式工程结构
ARCHIVED	节点已归档，不再作为当前执行对象

状态变化必须更新：

- 00_NODE_STATUS.md ；
- 00_PACKAGE_MANIFEST.json ；
- decisions/decision_log.md。

十九、版本规范

推荐版本格式：

v0.1
v0.2
v0.3
v1.0

使用建议：

- v0.x：草稿、实验、早期结构；
- v1.0：首次稳定规范；
- 小修改更新小版本；
- 重大结构变化更新主版本；
- 每次版本变化必须写入 Decision Log。

压缩包命名推荐：

NEM_Node_Package_<node_name>_<version>.zip

通用模板包命名推荐：

NEM_Node_Package_Format_Standard_Template.zip

二十、命名规范

目录命名

- 使用小写英文；
- 多词使用下划线或短横线；
- 保持稳定；
- 不使用空格；
- 不使用过长名称。

文件命名

推荐格式：

```
readable_overview.pdf
node_definition.md
active_aep.md
current_gate.md
package_rules.md
ai_execution_rules.md
config.template.json
report.template.md
decision_log.md
```

不推荐

```
最终版.pdf
新版2.md
这个给AI看.txt
临时文件.md
复制文件.md
```

原因：这类文件名不适合长期工程化，也不适合 AI 稳定读取。

二十一、AI 执行规则

AI 接手压缩包后，应遵守以下规则：

- 先读 00_README_AI.md；
- 再读 Manifest；
- 再读 Status；
- 再读 Rules；
- 再读当前 Gate；
- 再读 active AEP；
- 不要跳过 Gate；

- 不要擅自重命名目录；
- 不要把模板当成真实配置；
- 不要删除历史日志；
- 不要用 PDF 作为唯一执行依据；
- 执行后必须更新状态、报告和决策记录；
- 遇到失败，先记录失败，再提出修复 AEP。

二十二、完成定义

一个标准 NEM 压缩包被认为合格，需要满足：

- 有人类可读 PDF；
- 有人类入口 README；
- 有 AI 接手 README；
- 有机器可读 Manifest；
- 有节点状态文件；
- 有节点总定义；
- 有 AEP 模板或当前 AEP；
- 有 Gate 验收文件；
- 有规则目录；
- 有模板目录；
- 有命令目录；
- 有报告和日志目录；
- 有决策日志；
- 有 Backlog；
- 目录结构清晰；
- 文件命名稳定；
- 没有具体任务编号依赖；
- 可以被复用于不同节点。

二十三、最终原则

NEM 压缩包的本质，是把一个工程节点封装成可理解、可执行、可验证、可升级的节点容器。

它不是单个文档。

它不是一次性任务。

它不是随手整理的资料夹。

它是 AI 原生工程体系中的一个标准化节点单元。

一句话定义：

NEM 节点压缩包 = 人类说明书 + AI 接手协议 + 节点定义 + 工程原子包 + 进化闸门 + 运行模板 + 报告日志 + 决策记忆。

这个格式一旦稳定，后续任何工程节点都可以按照同一套结构生成、交接、执行、验收和演化。